

# Loanliness: Predicting Loan Repayment Ability with Machine Learning

**ML Mini-Project Report** · UE24CS352A – Machine Learning **Students:** PES1UG24CS149, PES1UG24CS170 **Reference paper:** *Loanliness: Predicting Loan Repayment Ability by Using Machine Learning Methods*

## 1. Abstract

This project implements the main workflow of the Loanliness reference paper on the Home Credit Default Risk dataset. Seven related data tables are aggregated into one applicant-level feature set, sparse features are removed, and the remaining data is encoded and normalised. Five classifiers (Logistic Regression, Random Forest, Naive Bayes, a Multi-Layer Perceptron and LightGBM) are trained on a down-sampled training set and compared using accuracy, precision, recall, F1 and ROC-AUC. We also compare three class-balancing strategies, test a polynomial feature transform, run K-Means clustering, visualise the data with PCA and t-SNE, and analyse Random Forest feature importance. LightGBM performed best (ROC-AUC 0.762, F1 0.270). Balancing the classes raised defaulter recall from 1.6% to about 68%, but ROC-AUC stayed near 0.75 for every strategy, which suggests the limit is in the information the features carry rather than in the model.

## 2. Problem Statement

The task is binary classification: given an applicant's current application and historical credit records, predict whether the applicant will repay the loan ( `TARGET = 0` ) or default ( `TARGET = 1` ). Two difficulties are highlighted by the reference paper:

1. The data contains many missing or invalid entries.
2. The classes are heavily imbalanced, so accuracy alone is a poor evaluation measure.

## 3. Dataset

The Home Credit Default Risk dataset has seven files: `application_train`, `bureau`, `bureau_balance`, `previous_application`, `POS_CASH_balance`, `installments_payments` and `credit_card_balance`.

| Table                 | Rows       | Columns |
|-----------------------|------------|---------|
| application_train     | 307,511    | 122     |
| bureau                | 1,716,428  | 17      |
| bureau_balance        | 18,829,755 | 3       |
| previous_application  | 344,265    | 37      |
| POS_CASH_balance      | 2,347,629  | 8       |
| installments_payments | 4,419,340  | 8       |
| credit_card_balance   | 2,452,181  | 23      |

The target is strongly imbalanced: 282,686 applicants (91.93%) repaid and 24,825 (8.07%) defaulted.

## 4. Methodology

### 4.1 Feature concatenation

Most history tables hold several rows per applicant, so each was aggregated to one row per `SK_ID_CURR` using the count, mean, minimum and maximum of every numeric column. `bureau_balance` is keyed by `SK_ID_BUREAU`, so it was aggregated at bureau level first and joined to `bureau`. The aggregated tables were left-joined to the application table, producing a combined dataset of 307,511 rows and 402 columns (386 numeric, 16 categorical, no duplicate rows).

### 4.2 Invalid and empty entries

Following the paper, any feature with more than 30% missing values was removed. This dropped 256 of 402 columns, leaving 146. The applicant ID and target were then separated from the inputs, giving 144 predictive features (133 numeric, 11 categorical). The worst offenders were the interest-rate columns of `previous_application` (99.7% missing) and several credit-card columns (about 80% missing).

### 4.3 Train-test split

A stratified 80/20 split with a fixed seed produced 246,008 training and 61,503 test rows, preserving the 92/8 class ratio in both sets.

### 4.4 Encoding and normalisation

A scikit-learn `ColumnTransformer` was used. Numeric features are mean-imputed and standardised; categorical features are imputed with the most frequent value and one-hot encoded. The pipeline was fit on training data only and then applied to the test data, which avoids leakage. The result has 239 features.

### 4.5 Class balancing

Three strategies from the paper were implemented:

- **Down-sampling:** the majority class was randomly reduced to the size of the minority class (19,860 each, 39,720 rows total).
- **SMOTE up-sampling:** synthetic minority samples brought both classes to 226,148 rows (452,296 total).
- **Class weights:** `class_weight="balanced"` applies inverse-frequency weights without changing the data.

The main model comparison uses the down-sampled training set, as the paper reports its best overall performance there.

### 4.6 Models and evaluation

| Model                | Configuration                                                      |
|----------------------|--------------------------------------------------------------------|
| Logistic Regression  | max_iter = 1000                                                    |
| Random Forest        | 200 trees                                                          |
| Gaussian Naive Bayes | default                                                            |
| MLP                  | hidden layers (128, 64), ReLU, Adam, early stopping, max_iter = 50 |
| LightGBM             | 200 estimators, learning rate 0.05, 31 leaves                      |

All models were evaluated on the untouched, imbalanced test set using accuracy, precision, recall, F1 and ROC-AUC, plus ROC curves and a confusion matrix for the best model. All random seeds were fixed at 42.

## 5. Results

### 5.1 Classifier comparison (down-sampled training data)

| Model               | Accuracy | Precision | Recall | F1     | ROC-AUC |
|---------------------|----------|-----------|--------|--------|---------|
| LightGBM            | 0.6967   | 0.1678    | 0.6961 | 0.2704 | 0.7617  |
| Logistic Regression | 0.6929   | 0.1642    | 0.6856 | 0.2650 | 0.7546  |
| MLP                 | 0.6964   | 0.1638    | 0.6729 | 0.2635 | 0.7486  |
| Random Forest       | 0.6966   | 0.1641    | 0.6735 | 0.2639 | 0.7453  |
| Naive Bayes         | 0.3388   | 0.0959    | 0.8528 | 0.1724 | 0.6450  |

LightGBM has the highest ROC-AUC and F1. The top four models lie within 0.016 AUC of one another, so the choice among them matters less than the data. Naive Bayes predicts most applicants as defaulters: its recall is highest (0.853) but its precision (0.096) and accuracy (0.339) are poor, likely because its feature-independence assumption is badly violated by the many correlated aggregate features. Precision is low for every model because defaulters are only 8% of the test set, so even a good recall produces many false alarms.

### 5.2 Effect of class balancing (Logistic Regression)

| Strategy                 | Accuracy | Precision | Recall | F1     | ROC-AUC |
|--------------------------|----------|-----------|--------|--------|---------|
| Original imbalanced data | 0.9195   | 0.5532    | 0.0157 | 0.0306 | 0.7545  |
| Class weights            | 0.6940   | 0.1643    | 0.6828 | 0.2648 | 0.7558  |
| Down-sampling            | 0.6929   | 0.1642    | 0.6856 | 0.2650 | 0.7546  |
| SMOTE                    | 0.6977   | 0.1640    | 0.6699 | 0.2635 | 0.7508  |

Trained on the original data, the model reaches 91.95% accuracy but identifies only 1.6% of defaulters, which is barely better than predicting "repaid" for everyone (91.93%). All three balancing methods lift recall to roughly 67–69% at the cost of accuracy. ROC-AUC

barely moves (0.751 to 0.756), showing that balancing shifts the decision threshold rather than improving how well the model ranks applicants. Class weights match down-sampling while using all the training data, so they are the more economical option.

### 5.3 Polynomial feature transformation

A degree-2 polynomial expansion was applied to a subset of 10 processed features, growing them to 65. Logistic Regression on this set scored accuracy 0.596, recall 0.605, F1 0.195 and ROC-AUC 0.643, worse than the full-feature model (0.755). The expansion was restricted to a subset because expanding all 239 features is computationally impractical, so this result shows the effect of using few features rather than a verdict on polynomial features generally.

### 5.4 K-Means clustering

The paper clusters borrowers and trains one classifier per cluster. We reduced a 30,000-row sample of the balanced training data to 20 principal components, examined  $k = 2$  to 6 with an elbow curve, and used  $k = 4$  following the paper. Cluster sizes were 5,075, 3,626, 12,251 and 9,048. One LightGBM model was trained per cluster. The notebook trains these models but does not report held-out metrics for them, so we cannot yet say whether cluster-specific modelling improves on the single global LightGBM.

### 5.5 PCA and t-SNE visualisation

The first two principal components of a 10,000-row sample explain only 23.8% of variance (12.3% and 11.4%). Both PCA and t-SNE (3,000 rows, with and without a PCA step) show the two classes strongly overlapping, with no clean separation. This is consistent with the modest ROC-AUC values: repayment ability is genuinely hard to separate with these features.

### 5.6 Feature importance

Random Forest importance over the 239 processed features ranks the following highest:

| Rank | Feature      | Importance |
|------|--------------|------------|
| 1    | EXT_SOURCE_2 | 0.0506     |
| 2    | EXT_SOURCE_3 | 0.0408     |
| 3    | DAYS_BIRTH   | 0.0229     |

| Rank | Feature           | Importance |
|------|-------------------|------------|
| 4    | DAYS_EMPLOYED     | 0.0195     |
| 5    | DAYS_ID_PUBLISH   | 0.0186     |
| 6    | DAYS_REGISTRATION | 0.0182     |
| 7    | AMT_CREDIT        | 0.0181     |
| 8    | AMT_ANNUITY       | 0.0180     |

The two external credit scores dominate, followed by age, employment and registration timing, and loan amounts. Aggregated bureau and POS-cash history features also appear in the top 20, which supports the value of the feature-concatenation step.

## 6. Comparison with the Reference Paper

| Classifier          | Paper accuracy | This project |
|---------------------|----------------|--------------|
| Logistic Regression | 69.34%         | 69.29%       |
| Random Forest       | 63.51%         | 69.66%       |
| Naive Bayes         | 52.11%         | 33.88%       |
| MLP                 | 62.34%         | 69.64%       |
| LightGBM            | 57.47%         | 69.67%       |

Logistic Regression reproduces the paper almost exactly. Our Random Forest, MLP and LightGBM score higher, and our Naive Bayes lower. Differences are plausible given our own aggregation statistics, one-hot encoding, hyperparameters and package versions, and the paper's figures should be treated as context rather than a target. The paper ranks LightGBM below Logistic Regression on accuracy; in our run LightGBM is first on AUC and F1 but only marginally ahead.

## 7. Discussion and Limitations

- **Accuracy is misleading** on this dataset; recall, F1 and ROC-AUC give a truer picture.

- **Precision stays near 0.16** for the stronger models. Using them to automatically reject applicants would turn away many good borrowers, so the output is better used as a risk score.
- **Hyperparameters were not tuned.** Gains from tuning LightGBM or the MLP were not explored.
- **Single split.** Results come from one train-test split with one seed, so small differences (such as between Random Forest and the MLP) are not statistically established.
- **Cluster-specific models** were trained but not evaluated on held-out data.
- **Polynomial features** were demonstrated on 10 features only.
- **Accuracy is threshold dependent** at the default 0.5 cutoff; threshold tuning was not performed.

## 8. Conclusion

Careful preprocessing and class balancing are essential for this problem. Removing sparse features reduced 402 columns to 144 before encoding, and balancing the classes turned a model that detected 1.6% of defaulters into one that detects about 68–70%. LightGBM was the best classifier (ROC-AUC 0.762, F1 0.270), with Logistic Regression a close and much simpler second. The PCA and t-SNE plots and the narrow AUC range across models indicate that the features set a ceiling of roughly 0.76 AUC. Feature importance shows that external credit scores and applicant age and employment history drive predictions most. Future work should tune the models, evaluate the cluster-specific classifiers, apply threshold tuning, and extend polynomial features to the full feature set.

## 9. References

1. *Loanliness: Predicting Loan Repayment Ability by Using Machine Learning Methods* (reference paper).
2. Home Credit Default Risk dataset (Kaggle competition data).
3. Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, 2011.
4. Ke et al., *LightGBM: A Highly Efficient Gradient Boosting Decision Tree*, NeurIPS 2017.
5. Chawla et al., *SMOTE: Synthetic Minority Over-sampling Technique*, JAIR 16, 2002.